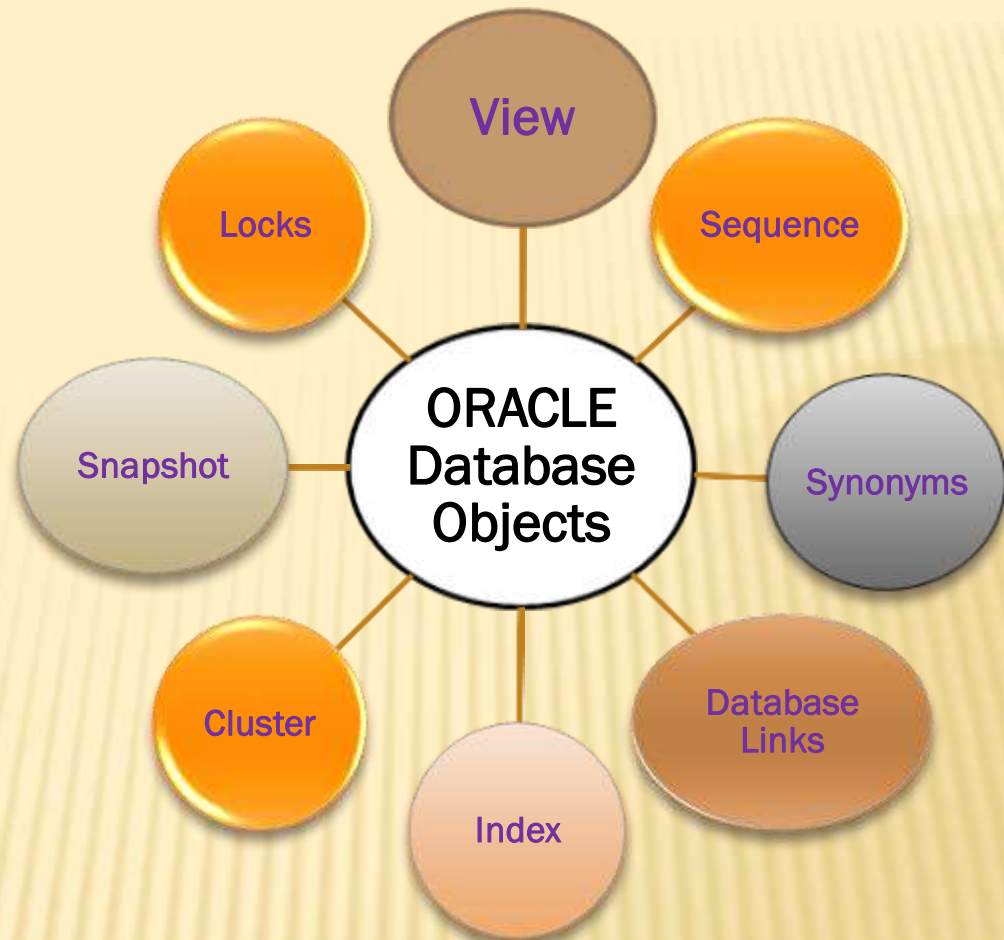




Unit - 3

**OTHER ORACLE DATABASE OBJECTS,
CONCURRENCY CONTROL USING LOCK**

TOPIC

- ✗ • View
- ✗ • Sequence
- ✗ • Synonyms
- ✗ • Database Links
- ✗ • Overview of Index and their types
- ✗ • Cluster
- ✗ • Snapshot
- ✗ • Locks, Overview of Locking Issues, Lock types

VIEW

VIEW

- × Definition

- × A **View** in SQL is a **virtual table** created from the result of a **SELECT** query.
- × It does not usually store the actual data separately; instead, it displays data from one or more underlying tables.

- × Simple definition:

- × A View is a virtual table based on an SQL query.

✕ Use of view

- + It simplifies complex SQL queries.
- + It provides data security by hiding unnecessary columns.
- + It makes frequently used queries easier to reuse.
- + It can combine data from multiple tables.
- + It provides a customized view of the data.

× Syntax view:

- × CREATE VIEW view_name AS
- × SELECT columns1,columns2,...
- × FROM table_name
- × WHERE condition;

```
× CREATE TABLE Student
× (
×     StudentID INT,
×     Name VARCHAR(50),
×     City VARCHAR(50),
×     Marks INT
× );

× INSERT INTO Student VALUES(1, 'Rahul', 'Ahmedabad', 85);
× INSERT INTO Student VALUES(2, 'Amit', 'Surat', 72);
× INSERT INTO Student VALUES(3, 'Neha', 'Ahmedabad', 90);
× INSERT INTO Student VALUES(4, 'Priya', 'Rajkot', 65);
× INSERT INTO Student VALUES(5, 'Karan', 'Ahmedabad', 78);

× SELECT * FROM STUDENT;
```

STEP BY STEP VIEW PRACTICAL

✖ STEP-1 (Create a Simple View)

- + CREATE VIEW Ahmedabad_Students AS
- + SELECT StudentID, Name, Marks
- + FROM Student
- + WHERE City = 'Ahmedabad';

✖ STEP-2 (View Display)

- + SELECT * FROM Ahmedabad_Students;

✖ STEP-3 (Specific Columns Select in View)

- + CREATE VIEW Student_Marks AS
- + SELECT Name, Marks
- + FROM Student;

✖ STEP-4 (View Display)

- + SELECT * FROM Student_Marks;

× EX.

× STEP(1)

+ create view emp14 as select ename,job,sal from emp

× STEP(2)

+ select *from emp14 (Display view table)

× STEP(3)

+ select *from emp (Display table)

+ **Drop view**

+ Drop view <view_name>

VIEW WITH WHERE CONDITION

- ✗ ONLY DISPLAY MARK 80 TO UP

- ✗ STEP-1 (Create view)

- + CREATE VIEW Top_Students AS
- + SELECT StudentID, Name, Marks
- + FROM Student
- + WHERE Marks >= 80;

- ✗ STEP-2 (Display view)

- + SELECT * FROM Top_Students;

SEQUENCE

SEQUENCE

- ✗ **What is a Sequence?**
- ✗ A **Sequence** in SQL is a database object that automatically generates a series of **numeric values** according to specified rules.
- ✗ It is commonly used to generate unique numbers for columns such as:
 - + Student ID
 - + Employee ID
 - + Order ID
 - + Invoice Number
 - + Product ID
- ✗ Primary key column to maintain unique values
- ✗ In multi user environment.
- ✗ **SEQUENCE = Automatic Number Generate Database Object.**

× Main Uses SEQUENCE

- + Generate unique numbers
- + Auto numbering
- + Generate IDs
- + Avoid duplicate numbers
- + Control starting number and increment

✗ Syntax:

+ Create SEQUENCE <name>

✗ [Increment by<no.>]

→ increase/decrease.

✗ [Start with <no.>]

→ first number
sequence.

✗ [Maxvalue <no>/nomaxvalues]

→ largest value seq.

✗ [Minvalue<no.>/nominvalue]

→ lower value.

STEP-BY-STEP SEQUENCE CREATION

✖ Step 1: Create Sequence

- + CREATE SEQUENCE student_seq
- + START WITH 1
- + INCREMENT BY 1;

✖ This creates a sequence named student_seq.

✖ The first generated value will be 1.

✖ Step 2: Generate the First Number

- + SELECT student_seq.NEXTVAL
- + FROM dual;

+ Output: 1

✖ NEXTVAL returns the next value of the sequence.

STEP-BY-STEP SEQUENCE CREATION

✖ Step 3: Generate Next Values

- + SELECT student_seq.NEXTVAL
- + FROM dual;

✖ Output: 2

✖ Step 4: Generate Next Values

- + SELECT student_seq.NEXTVAL
- + FROM dual;

✖ Output: 3

✖ Step 5: Generate Next Values

- + SELECT student_seq.NEXTVAL
- + FROM dual;

✖ Output: 4

✖ Sequence Flow

- + NEXTVAL → 1
- + NEXTVAL → 2
- + NEXTVAL → 3
- + NEXTVAL → 4
- + NEXTVAL → 5
- + ...

✖ What is a **Current Value**?

+ CURRVAL returns the current value that was most recently generated by NEXTVAL in your session.

✖ SELECT student_seq.CURRVAL FROM dual;

✖ SELECT student_seq.CURRVAL FROM dual;

✖ SELECT student_seq.CURRVAL FROM dual;

✖ Output: 5

✖ Output: 6

✖ Output: 7

✖ **NEXTVAL** = Next number

✖ **CURRVAL** = Current/last generated number

START WITH / INCREMENT BY / MAXVALUE

× Ex. STEP(1)

+ create table stud (id number(5),name varchar2(10),city varchar2(10))

× STEP(2)

+ create sequence seq01

× increment by 20

start with 10

× maxvalue 100

× STEP(3)

+ insert into stud values(seq01.nextval,'ccc','jnd');

× STEP(4)

+ select *from stud;

+ Drop sequence

× Drop sequence < sequence _name>

INDEX

INDEX

What is an Index?

- ✘ An **Index** in SQL is a database object that helps the database **find and retrieve data faster** from a table.
- ✘ It works similarly to an **index in a book**.
- ✘ Instead of reading every page to find a topic, you can use the book's index to quickly locate it.
- ✘ **INDEX** = speed Data search in database structure.

✖ Why do we need Index?

✖ Ex. Student table 10 lakh (1 million) records.

✖ Student

✖ -----

StudentID	Name	City	Marks
1	Rahul	Ahmedabad	85
2	Amit	Surat	72
3	Neha	Rajkot	90
...			
1000000	XYZ	Surat	75

✖ `SELECT * FROM Student WHERE StudentID = 600000;`

✖ The database may need to examine many rows to find the required record.

✖ Advantage — Faster Searching

+ Indexes can make suitable search queries faster.

SIGNAL COLUMN INDEX

- ✗ Create an Index

- + CREATE INDEX idx_student_name
 - + ON Student(Name);

- ✗ idx_student_name is the name of the index.
- ✗ The index is created on the Name column of the Student table.

- + Select * from idx_student_name;

MULTIPLE COLUMN INDEX / COMPOSITE INDEX

- ✖ An index created on **two or more columns** is called a **Composite Index** or **Multi-Column Index**.

- ✖ **Ex.**
 - + CREATE INDEX idx_city_marks
 - + ON Student(City, Marks);

- ✖ **Query:**
 - + SELECT *
 - + FROM Student
 - + WHERE City = 'Ahmedabad'
 - + AND Marks = 85;

✖ Advantages of Index

- ✖ Advantage 1 — Faster Searching
- ✖ Advantage 2 — Better Query Performance
- ✖ Advantage 3 — Useful for Large Tables
- ✖ Advantage 4 — Helps with Sorting

✖ Disadvantages of Index

- ✖ Disadvantage 1 — Extra Storage
- ✖ Disadvantage 2 — INSERT Performance
- ✖ Disadvantage 3 — UPDATE Performance
- ✖ Disadvantage 4 — DELETE Performance

SYNONYM

SYNONYM

- ✗ What is a Synonym?
- ✗ A **Synonym** in SQL is an **alternative name or alias** for a database object.
- ✗ It can be created for objects such as:
 - + Table
 - + View
 - + Sequence
 - + Procedure
 - + Function
- ✗ Other supported database objects
- ✗ Instead of using the original object name, we can use the Synonym name.

✕ Why do we use Synonym?

+ Synonyms are used to:

- ✕ Provide a shorter object name.
- ✕ To hide the actual object name.
- ✕ Make SQL statements easier to write.
- ✕ Provide an alternative name for an object.
- ✕ Simplify access to objects in other schemas when permissions are available.

✖ Ex.

+ Step-1

✖ create synonym syn_123 for emp

+ Step-2

✖ select *from syn_123

+ Step-3

+ Drop synonym

+ Drop **synonym** < **synonym _name**>

✖ drop synonym syn_123

+ Step-4

✖ select *from emp

DATABASE LINK

DATABASE LINK

- ✖ Database link allows you to connect from one database to another.
- ✖ You can establish the direct connection from local database to remote database.
- ✖ A database link can be private –owned by a single user, or public in which all the user in the local database can use the link.

✖ Ex.

✖ create database link myremotedb using
'system'

✖ select sysdate from dual myremotedb

CLUSTER

CLUSTER

✖ 1.What is a Cluster in Oracle?

- + A **Cluster** is a database object that allows related data from **multiple tables** to be stored physically close together in the database.
- + (Cluster એ Oracle database નો એવો object છે જેમાં related tables નો data physically નજીક-નજીક store કરી શકાય છે.)

✖ 2. Why do we use Cluster?

- + Clusters are mainly used to improve performance when two or more tables are frequently queried together using the same column.
 - + (જ્યારે બે અથવા વધારે tables ને વારંવાર એક જ column દ્વારા JOIN કરવામાં આવે ત્યારે Cluster ઉપયોગી થઈ શકે છે.)
- ✖ A cluster is a schema object that contains data from one or more tables. All of which have one or more column is common.
- ✖ Table clustering is very seldom used by oracle DBA's and developers.

REAL-LIFE EXAMPLE

✕ Cluster Create Process

- + 1. Create Cluster
- + ↓
- + 2. Create Tables using Cluster
- + ↓
- + 3. Create Cluster Index
- + ↓
- + 4. Insert Data
- + ↓
- + 5. Query Data

✖ Step 1 — Create Cluster

```
+ CREATE CLUSTER college_cluster  
+ (  
+   course_id NUMBER(3)  
+ );
```

✖ Step 2 — Create Tables Student Using Cluster

```
+ CREATE TABLE student  
+ (  
+   student_id NUMBER,  
+   student_name VARCHAR2(30),  
+   course_id NUMBER(3)  
+ )  
+ CLUSTER college_cluster(course_id);
```

✖ Step 3 — Create COURSE Table

```
+ CREATE TABLE course  
+ (  
+   course_id NUMBER(3),  
+   course_name VARCHAR2(30)  
+ )  
+ CLUSTER college_cluster(course_id);
```

✖ Step 4 — Create Cluster Index

- + CREATE INDEX college_cluster_idx
- + ON CLUSTER college_cluster;

- ✖ Cluster
- ✖ ↓
- ✖ Cluster Index
- ✖ ↓
- ✖ Find Cluster Key
- ✖ ↓
- ✖ Find related rows

✖ Step 5 — Insert Data (COURSE TABLE)

- ✖ INSERT INTO course VALUES (101, 'BCA');
- ✖ INSERT INTO course VALUES (102, 'BCOM');
- ✖ INSERT INTO course VALUES (103, 'BA');

✖

✖ Insert Data (STUDENT TABLE)

- ✖ INSERT INTO student VALUES (1, 'Rahul', 101);
- ✖ INSERT INTO student VALUES (2, 'Amit', 102);
- ✖ INSERT INTO student VALUES (3, 'Neha', 101);
- ✖ INSERT INTO student VALUES (4, 'Priya', 103);

✖ Step 6 — Query

- + SELECT *FROM student;
- + SELECT *FROM course;

✖ Step-7 Cluster Actual Benefit

✖ JOIN

- + SELECT s.student_name,c.course_name
- + FROM student s, course c
- + WHERE s.course_id = c.course_id;

✖ Output:

★ STUDENT_NAME	COURSE_NAME
★ -----	
★ Rahul	BCA
★ Amit	BCOM
★ Neha	BCA
★ Priya	BA

SNAPSHOT

SNAPSHOT

✖ 1. What is Snapshot?

- + A **Snapshot** is a stored copy of data or the result of a query at a particular point in time.
- ✖ It is a read only copy of a table.
- ✖ A snapshot is more useful in distributed computing environment .that time using command create snapshot.
- ✖ Oracle's older terminology:
 - + **Snapshot = Materialized View**

× 2. Why do we use Snapshot?

+ Snapshots are mainly used for:

- × Data replication
- × Reporting
- × Historical data
- × Reducing database load
- × Storing query results
- × Data warehouse applications

× **STEP-1 First Create a Table (STUDENT table)**

× CREATE TABLE student
× (
× student_id NUMBER,
× student_name VARCHAR2(30),
× city VARCHAR2(30)
×);

× **STEP-2 Insert Data**

× INSERT INTO student VALUES (1, 'Rahul', 'Amreli');
× INSERT INTO student VALUES (2, 'Amit', 'Rajkot');
× INSERT INTO student VALUES (3, 'Neha', 'Surat');

× **STEP-3 Check Data**

× STUDENT_ID STUDENT_NAME CITY

× -----

× 1	Rahul	Amreli
× 2	Amit	Rajkot
× 3	Neha	Surat

✖ STEP-4 Create Snapshot

- + CREATE MATERIALIZED VIEW student_snapshot
- + AS
- + SELECT *
- + FROM student;

✖ STEP-5 Check Snapshot

- + SELECT * FROM student_snapshot;

+ OUTPUT:

✖ STUDENT_ID	STUDENT_NAME	CITY
✖ -----		
✖ 1	Rahul	Amreli
✖ 2	Amit	Rajkot
✖ 3	Neha	Surat

- ✖ **STEP-6 Important Concept — Original Table Change**
 - + UPDATE student SET city = 'Bhavnagar' WHERE student_id = 1;
 - + Select *from student:
 - + **Output:**
 - ✖ 1 Rahul Bhavnagar
 - ✖ 2 Amit Rajkot
 - ✖ 3 Neha Surat

- ✖ **STEP-7 (CHECK SNAPSHOT)**
 - ✖ **SELECT * FROM student_snapshot;**

- ✖ **STEP-8 OLD DATA DISPLAY**
 - ✖ 1 Rahul Amreli
 - ✖ 2 Amit Rajkot
 - ✖ 3 Neha Surat

- ✖ **STEP-9 Refresh Snapshot**
 - ✖ **BEGIN**
 - ✖ **DBMS_MVIEW.REFRESH('STUDENT_SNAPSHOT');**
 - ✖ **END;**
 - ✖ **/**

- ✖ **STEP-10 (CHECK SNAPSHOT)**
 - ✖ **SELECT * FROM student_snapshot;**

- ✖ **STEP-11 NEW DATA DISPLAY**
 - ✖ 1 Rahul Bhavnagar
 - ✖ 2 Amit Rajkot
 - ✖ 3 Neha Surat

Thank you
?